

프로젝트 진행 업데이트 (1차) - 최종 수정본

Java 웹 기반 계절별 식물 관측 지도 시각화 시스템

항목	내용
최종 방향	Java 내장 HttpServer와 React(CDN)를 사용한 웹 지도 시각화
수정 이유	지도 표현과 사용자 접근성은 웹이 유리하므로 웹 화면을 채택하되, 서버/데이터 처리는 Java 클래스로 직접 구현
핵심 기능	식물/날짜 선택, Google Maps 위 관측지점 라벨링, 단계별 색상 표시, CSV 데이터 기반 API 응답
구현 규모	8개 패키지, 26개 클래스, 7개 인터페이스, 5개 enum, 추상 클래스 1개

요약

본 프로젝트는 계절별 대표 식물의 관측 상태를 한국 지도 위에 표시하는 Java 웹 애플리케이션이다. 웹 서버는 Spring 같은 프레임워크가 아니라 Java 표준 라이브러리의 HttpServer를 사용하여 구현한다. 화면은 React(CDN), CSS, JavaScript로 구성하고, 관측지점·관측기록·평년값은 CSV 파일로 저장한다. 따라서 웹 화면을 사용하더라도 핵심 데이터 흐름과 API 응답은 Java 클래스가 직접 제어한다.

- 대표 식물: 매화, 벚나무, 코스모스, 단풍나무
- 지도 표시: Google Maps JavaScript API 위에 관측지점 좌표를 라벨과 마커로 표시
- 서버 방식: com.sun.net.httpserver.HttpServer 기반 경량 웹 서버
- 관측 데이터: 1차 구현은 샘플 CSV 기반이며 실시간 공공데이터 연동은 향후 범위

1. 피드백 반영 및 주제 수정

초기에는 바닐라 Java 데스크톱 앱을 고려했으나, 지도 기반 시각화는 브라우저 화면에서 더 자연스럽게 사용자 접근성도 좋다. 따라서 최종 방향은 웹으로 변경한다. 다만 Java Programming 과목의 목표를 유지하기 위해 Java가 서버, CSV 로딩, 도메인 객체 생성, JSON 응답을 직접 담당하도록 설계한다.

1-1. 최종 반영 사항

- 웹 서버를 구현하되 외부 프레임워크 의존도를 낮춘다.
- 지도와 라벨링은 Google Maps JavaScript API와 React 컴포넌트에서 처리한다.
- 관측 데이터 관리는 Java Repository와 Service 클래스가 담당한다.
- DB는 1차 범위에서 제외하고 CSV 저장 방식으로 명확히 제한한다.
- API 호출과 데이터 흐름을 문서에 구체적으로 반영한다.

구분	이전안	최종안
화면	Java Swing	웹 브라우저
서버	없음 또는 데스크톱 내부 처리	Java HttpServer
지도 표현	Swing 그래픽	Google Maps JavaScript API
데이터 처리	Java 파일 I/O	Java Repository + API JSON 응답

2. 주요 사용자 시나리오

사용자는 브라우저에서 식물 종류와 조회 날짜를 선택한다. 서버는 CSV 파일에서 관측 데이터와 지점 정보를 읽고, 선택 조건에 맞는 최신 관측 상태를 JSON으로 반환한다. 화면은 반환된 좌표와 상태값을 이용해 지도 위에 마커와 라벨을 그린다.

2-1. 일반 사용자

- 사용자가 웹 페이지에 접속한다.
- 식물 종류를 선택한다. 예: 벚나무
- 조회 날짜를 선택한다.
- 조회 버튼을 누르면 /api/map 요청이 발생한다.
- 지도 위에 관측지점별 단계, 평년 대비 결과, 관측일이 표시된다.

2-2. 데이터 관리자

- stations.csv에 관측지점 코드, 이름, 위도, 경도, 주소를 등록한다.
- observations.csv에 식물별 관측 날짜와 단계를 등록한다.
- normal_years.csv에 평년 기준일을 등록한다.
- 필요하면 CSV를 수정한 뒤 서버를 재시작하여 데이터를 갱신한다.

3. 데이터 흐름

단계	입력	Java 처리	출력
1	웹 요청 /api/plants	PlantType enum 조회	식물 목록 JSON
2	웹 요청 /api/map	plantType, date 파라미터 파싱	조회 조건
3	stations.csv	CsvStationRepository	관측지점 객체 목록
4	observations.csv	CsvObservationRepository	관측기록 객체 목록
5	normal_years.csv	CsvNormalYearRepository	평년값 객체 목록
6	관측기록 + 평년값	ObservationMapService, ComparisonService	MapViewModel
7	MapViewModel	JsonSupport	지도 표시용 JSON
8	JSON 응답	React app.js	지도 마커/라벨 렌더링

3-1. API 엔드포인트

```
GET /api/plants
GET /api/map?plantType=CHERRY_BLOSSOM&date;=2026-04-06
```

4. 패키지 및 클래스 설계

패키지	역할	예상 클래스
seasonal.domain	도메인 객체	GeoPoint, ObservationStation, ObservationRecord, MapMarker, MapViewModel
seasonal.enums	상수 타입	PlantType, SeasonType, PhenologyStage, DataSourceType
seasonal.repository	CSV 저장소	StationRepository, CsvStationRepository, ObservationRepository
seasonal.service	비즈니스 로직	ObservationMapService, ComparisonService, JsonSupport
seasonal.calculator	상태 계산/상속	AbstractPhenologyStatusCalculator, 두 계산기 구현체
seasonal.map	지도 공급자/좌표 변환	GoogleMapsProvider, MapProjection, WebMercatorProjection
seasonal.config	키/아이콘 설정	ApiKeyConfig, IconRuleRepository
seasonal.web	웹 서버/API	SeasonalPlantWebServer

4-1. 설계 수량

구분	수량	비고
패키지	8개	도메인, 저장소, 서비스, 웹, 지도, 설정, 예외 등
클래스	26개	일반 클래스 25개와 abstract class 1개 구현
인터페이스	7개	Repository, Provider, Projection 등 역할 분리
enum	5개	식물, 계절, 단계, 데이터 출처, 비교 결과

5. 클래스/인터페이스 상세 설계

5-1. 주요 클래스

클래스	역할
GeoPoint	위도와 경도를 표현하는 값 객체
ObservationStation	관측지점 코드, 이름, 좌표, 주소 관리
ObservationRecord	식물별 관측 날짜, 단계, 출처 저장
NormalYearReference	관측지점별 평년 기준일 저장
MapMarker	지도 위에 표시할 지점별 상태 정보
MapViewModel	화면에 전달할 지도 데이터 묶음
SeasonalPlantWebServer	정적 페이지와 API를 제공하는 Java 웹 서버

5-2. 인터페이스

- StationRepository: 관측지점 조회
- ObservationRepository: 관측기록 조회
- NormalYearRepository: 평년값 조회
- MapProvider: 외부 지도 API 이미지 제공
- MapProjection: 좌표를 화면 위치로 변환
- PhenologyStatusCalculator: 식물 상태 계산
- IconRuleRepository: 상태별 아이콘 규칙 조회

5-3. 상속 구조

AbstractPhenologyStatusCalculator를 두고, FloweringPlantStatusCalculator와 FoliagePlantStatusCalculator가 이를 상속하여 개화형 식물과 단풍형 식물의 계산 방식을 분리한다.

6. 저장 방식

1차 구현에서는 DB를 사용하지 않는다. 데이터 규모가 작고 제출 시 로컬 실행이 중요하므로 CSV 저장 방식이 적합하다. 다만 Repository 인터페이스를 사용하므로 이후 DB 구현체로 교체할 수 있다.

파일	저장 형식	주요 필드
src/main/resources/data/stations.csv	CSV	stationCode, name, latitude, longitude, address
src/main/resources/data/observations.csv	CSV	stationCode, plantType, observedDate, stage, sourceType
src/main/resources/data/normal_years.csv	CSV	stationCode, plantType, normalDate
src/main/resources/web/index.html	HTML	웹 화면 구조
src/main/resources/web/app.js	React/JavaScript	API 요청과 지도 마커 렌더링

6-1. 저장 방식 선택 이유

- 단일 사용자 과제 프로젝트라 DB 서버 설정이 필수적이지 않다.
- CSV는 관측지점과 관측기록처럼 표 형태 데이터 관리에 적합하다.
- Repository 패턴을 적용해 DB 확장 가능성을 열어 둔다.

7. 지도 표시 및 API 설계

지도는 Google Maps JavaScript API로 표시한다. Java 서버는 관측지점의 위도/경도와 상태값을 JSON으로 전달하고, React 프론트엔드는 Google Maps 객체 위에 AdvancedMarkerElement를 이용한 단계별 색상 마커와 지점 라벨을 올린다.

7-1. 지도 라벨링 방식

- 서버가 관측지점의 위도/경도와 식물 상태를 JSON으로 전달한다.
- Google Maps가 위도/경도를 지도 화면 위치로 변환한다.
- 각 마커는 단계별 색상과 지점명 라벨을 가진다.
- 마커를 클릭하면 지점명, 단계, 관측일, 평년 대비 결과를 확인할 수 있다.

7-2. API 키 관리

Google Maps 키는 웹 리소스에 직접 저장하지 않는다. Java의 ApiKeyConfig가 GOOGLE_MAPS_API_KEY 환경 변수 또는 config/google-maps.properties에서 키를 읽고, /config.js 응답을 동적으로 생성한다. 실제 키 파일은 .gitignore에 포함하며 Google Cloud에서 HTTP referrer 제한을 설정한다.

8. 멀티스레드 및 예외 처리

Java `HttpServer`는 `Executor`를 통해 요청을 병렬 처리한다. 현재 서버는 고정 크기 스레드 풀을 사용하므로 여러 사용자가 동시에 API를 호출해도 각 요청을 독립적으로 처리할 수 있다.

항목	설계 내용
요청 처리	<code>HttpServer</code> 와 8개 <code>daemon worker thread</code> 사용
파일 로딩 오류	<code>DataLoadException</code> 으로 변환
잘못된 요청	400 JSON 오류 응답
지원하지 않는 메서드	405 <code>Method Not Allowed</code> 응답

8-1. 예외 처리 원칙

- CSV 파일 누락 또는 파싱 오류는 서버 내부 예외로 처리한다.
- 잘못된 `plantType` 또는 날짜 형식은 400 응답으로 반환한다.
- 프론트엔드는 오류 메시지를 사용자에게 표시한다.

9. 구현 현황 및 테스트 결과

구분	현재 상태	검증 방법
Java 서버	SeasonalPlantWebServer 구현	브라우저에서 localhost:8080 접속
API	/api/plants, /api/map 구현	JSON 응답 확인
데이터	stations, observations, normal_years CSV 작성	Repository 자동 테스트
화면	React 기반 index.html, styles.css, app.js 작성	식물/날짜 선택 후 지도 표시 확인
상속	추상 계산기와 개화형/단풍형 구현	계산기 선택·날짜 경계 테스트
자동 테스트	6개 테스트 클래스, 10개 테스트 케이스	./gradlew test 통과

9-1. 결론

최종 방향은 Java 웹 기반 지도 시각화 시스템이다. Google Maps는 실제 지도 API를 사용하고, 식물 관측값은 현재 샘플 CSV를 사용한다. 핵심 데이터 처리와 API 응답은 Java 클래스가 담당하므로 객체지향 설계, 파일 입출력, enum, 인터페이스, 상속, 예외 처리, 멀티스레드 학습 요소를 반영한다.